

EVALUACIÓN FINAL – 2026-2

Unholy Store

Plataforma web de administración y comercio con consumo de API REST, autenticación, carrito de compras y diseño responsivo.

CARRERA

Ingeniería de software

ASIGNATURA

Desarrollo Web

DOCENTE

Yan Carlo Angarita

CIUDAD Y FECHA

Los Patios, 20 de septiembre de 2026

Integrantes del equipo

Dumas Alejandro Restrepo Moreno

https://github.com/DumasRestrepo/parcial_desarrollo_web

Diego Andrés Sanabria Ascanio

<https://github.com/Mort0/Parcial-desarrollo-web>

El código fuente evaluable corresponde a la aplicación Unholy Store, desarrollada en React, TypeScript y Vite, con persistencia sobre Mock API.

1. Descripción del proyecto

Unholy Store es una aplicación de una sola página (SPA) para una tienda de moda urbana. El sistema combina un catálogo de productos con un panel administrativo: permite gestionar el inventario, los clientes, las órdenes y la información institucional, e incorpora un flujo de compra para usuarios autenticados. La persistencia de datos se realiza contra un Mock API REST, sin backend propio.

El trabajo parte de la entrega del parcial y completa el ciclo pedido en la evaluación final: CRUD de todos los recursos, sesión con rutas protegidas, carrito, creación persistente de órdenes y retroalimentación visual al usuario.

2. Arquitectura

La solución sigue una arquitectura en capas en el frontend. La interfaz no habla con la API de forma directa: cada recurso pasa por un repositorio y un servicio HTTP único. El estado de sesión, carrito, alertas e información institucional se comparte con Context API.

PRESENTACIÓN

Páginas React (catálogo, CRUDs, login, información), Layout, Header, Footer y CartDrawer. Enrutamiento con React Router y *basename* para despliegue en GitHub Pages.

ESTADO DE LA APLICACIÓN

AuthContext (login, registro, logout y rol), CartContext (ítems, cantidades y persistencia local), AlertContext (toasts y confirmaciones), InformationContext (misión, visión y contacto) y ThemeContext (paletas).

ACCESO A DATOS

Repositorios por recurso: producto, categoria, usuario, cliente, orden, estado_orden e information. Cada uno expone `getAll`, `getById`, `create`, `update` y `remove`.

INFRAESTRUCTURA

`api.service.ts` encapsula `fetch` sobre `https://6aa6bdb0d7765db9850793a1.mockapi.io` con métodos GET, POST, PUT y DELETE.

Stack

- React 19 y TypeScript
- Vite 8
- React Router DOM 7
- MockAPI como backend REST

Seguridad de rutas

- Públicas: `/login` y `/register`
- El resto exige sesión activa
- Sin sesión hay redirección a login
- `/usuarios` queda restringido a rol admin

3. Alcance implementado

El alcance cubre los cuatro bloques de la evaluación final. Lo siguiente resume lo que el docente puede verificar en el código y en la aplicación en ejecución.

Consolidación del parcial y CRUD

Existen vistas y operaciones de crear, listar, actualizar y eliminar para productos, categorías, usuarios, clientes, órdenes, estados de orden e información institucional. La página de información y el pie de página cargan misión, visión, teléfono, dirección y horario desde el endpoint */information*.

Autenticación y sesión

El login valida nombre y clave contra el recurso */usuario*. El encabezado muestra el usuario activo y el botón Cerrar sesión. Las vistas administrativas, el historial de pedidos y el checkout viven detrás de ProtectedRoute.

Tienda, carrito y pedidos

El catálogo permite explorar productos y agregarlos al carrito, modificar cantidades y quitar ítems. El carrito se guarda en localStorage de forma opcional. Al finalizar la compra, el sistema exige un usuario autenticado y envía un POST a */orden* con productos, cantidades, montos, fecha, estado inicial «Pendiente» y referencia al comprador (nombre y usuarioid). Después se vacía el carrito, se muestra un resumen y el pedido queda en el historial del usuario.

Diseño responsivo y feedback

La interfaz se adapta a escritorio y móvil (menú hamburguesa, grillas y pie de página). Hay toasts de éxito o error en login, operaciones CRUD, alta al carrito y confirmación de la orden.

4. Repositorios personales de GitHub

Conforme al enunciado, cada integrante mantiene el código en un repositorio personal. Los enlaces evaluables son:

INTEGRANTE	ENLACE DEL REPOSITORIO
Dumas Alejandro Restrepo Moreno	https://github.com/DumasRestrepo/parcial_desarrollo_web
Diego Andrés Sanabria Ascanio	https://github.com/Mort0/Parcial-desarrollo-web

El proyecto se desarrolló en el repositorio de Dumas Restrepo. Diego Sanabria tiene el suyo con la misma versión del código, como pide la evaluación a cada integrante.